A PRACTICAL GUIDE · REAL EXAMPLES

Git for DV Engineers

The Git skills that make a verification engineer fast: finding which change broke a regression, reproducing any failure exactly, and working cleanly in a team. Every example is a real session on a small FIFO verification project, with real simulations (Icarus Verilog 12, Git 2.43).

01 Why Git skills matter in DV
02 The example project
03 Set up Git once
04 What goes in the repo, and what never does
05 Reproduce any result exactly
06 Find what broke the regression
07 Who changed this line, and why
08 Fix it on a branch and open a Merge Request
09 Commit only what you mean: git add -p
10 Interrupted? git stash
11 Stay in sync and resolve testlist conflicts
12 Release branches, cherry-pick and tags
13 Undo safely, and the reflog rescue
14 Search the code fast: git grep
15 Team-scale topics
16 If your team uses Perforce
17 Interview questions you can now answer
18 Daily cheat sheet

01 Why Git skills matter in DV

In software, Git is mostly a place to save code. In verification it is also a **debugging tool**. When last night's regression passed and tonight's fails, Git tells you exactly what changed in between, finds the guilty commit for you, and lets anyone reproduce the failure on the exact same code.

Top DV teams look for this:

- Google's posting for a *Design Verification Engineer, TPU* (Bengaluru) lists "Experience with industry-standard simulators, **revision control systems**, and **regression systems**" as a preferred qualification.
- NVIDIA's posting for a *Senior ASIC Verification Engineer, GPU* lists "**Expert debugging** and analytical skills" as a way to stand out.

After this guide you will be able to:

SKILL	SECTION
Keep a DV repo clean: what to track, what to ignore	4
Reproduce any result exactly: commit + test + seed	5
Find the commit that broke a test with `git bisect`	6
Find who changed a line and why	7
Fix on a branch, open a Merge Request, commit only what you mean	8–9
Handle interruptions, conflicts, release branches and mistakes	10–13
Answer Git questions in DV interviews	17

> Some chip companies use Perforce or in-house tools instead of, or next to, Git. The ideas in this guide are the same everywhere; section 16 maps them to Perforce.

02 The example project

A small but real DV project: a 4-entry FIFO, a self-checking testbench and a regression script.

```
fifo_dv/
├── rtl/fifo.sv            design under test: 4-entry synchronous FIFO
├── tb/tb_fifo.sv          self-checking testbench; pick a test with +TEST=<name>
├── tests/testlist         the regression: one test name per line
├── scripts/run_test.sh    compile + run one test, print PASS/FAIL and the commit
├── .gitignore             keeps simulation output out of Git
└── README.md
```

The team and its history:

DATE	WHO	WHAT HAPPENED
12–15 Oct	Priya, Karthik, you	RTL, testbench, run script and 3 tests written
15 Oct night	Nightly regression	3/3 pass, tagged `nightly_pass_1015`
16–17 Oct	Karthik, Priya	4 more commits, including "rtl: simplify full flag logic"
17 Oct night	Nightly regression	`fifo_full` FAILS, tagged `nightly_fail_1017`
18 Oct	You	Find the bug, fix it, add a new test, ship the fix to the release branch

The run script is the key to everything that follows. It writes the exact commit into every log, and its exit code tells Git whether the test passed:

```
#!/usr/bin/env bash
# Usage: scripts/run_test.sh <test> [seed]
# Exit codes: 0 = pass, 1 = fail, 125 = could not build (git bisect skips it)
set -u
test=${1:?usage: run_test.sh <test> [seed]}
seed=${2:-1}
root=$(git rev-parse --show-toplevel) && cd "$root" || exit 125
mkdir -p sim/build sim/logs
log="sim/logs/${test}_${seed}.log"
rev=$(git describe --always --dirty --tags)          # which code is this?
printf 'commit: %s\ntest: %s  seed: %s\n' "$rev" "$test" "$seed" > "$log"
iverilog -V 2>&1 | head -n 1 >> "$log"               # which simulator version?
iverilog -g2012 -o sim/build/simv rtl/fifo.sv tb/tb_fifo.sv >> "$log" 2>&1 || { echo "BUILD FAILED
($rev)"; exit 125; }
vvp -n sim/build/simv +TEST="$test" +SEED="$seed" ${EXTRA:-} >> "$log" 2>&1
if grep -q "TEST PASSED" "$log"; then
  echo "PASS  $test  @ $rev"; exit 0
else
  echo "FAIL  $test  @ $rev   (log: $log)"; exit 1
fi
```

03 Set up Git once

These settings save time every day. Run them once on each machine you work on.

```
$ git config --global user.name "Abishek Kumar"
$ git config --global user.email "abishek@example.com"
$ git config --global core.editor vim
$ git config --global pull.rebase false
$ git config --global rerere.enabled true
$ git config --global alias.st "status -sb"
$ git config --global alias.lg "log --oneline --graph --decorate --all"
```

SETTING	WHY
`user.name`, `user.email`	Written into every commit. Use your real name and work email
`core.editor vim`	Commit messages and merge messages open in Vim
`pull.rebase false`	Newer Git refuses a `git pull` on diverged branches until you choose; this says "merge"
`rerere.enabled true`	**Re**use **re**corded **re**solution: Git remembers how you fixed a conflict and fixes the same conflict for you next time
`alias.st`, `alias.lg`	`git st` = short status; `git lg` = the whole history as a graph

Two more worth adding: `git config --global push.autoSetupRemote true` (Git 2.37 or newer: the first push of a new branch no longer needs `-u`), and `git config --global credential.helper 'cache --timeout=36000'` (remember your GitLab token for 10 hours).

04 What goes in the repo, and what never does

```
$ git clone https://gitlab.example.com/dv/fifo_dv.git
Cloning into 'fifo_dv'...
$ cd fifo_dv
$ git lg
* 33dfeb9 (HEAD -> main, tag: nightly_fail_1017, origin/release/a0, origin/main, origin/HEAD) rtl: add port comments
* 64ef749 scripts: record simulator version in each log
* 7c75b56 rtl: simplify full flag logic
* ade5d98 docs: add README with run instructions
* 097ad87 (tag: nightly_pass_1015) tb: add fifo_empty test
* 95e290c tb: add fifo_full test
* 042d8b3 scripts: add run_test.sh, testlist and .gitignore
* 5ead26d tb: add self-checking testbench with fifo_basic test
* 43d90c9 rtl: add synchronous fifo
```

Reading `git lg`: `HEAD -> main` is where you are, `tag: nightly_fail_1017` marks the code last night's regression ran on, `origin/release/a0` is the release branch on GitLab.

```
$ git ls-files
.gitignore
README.md
rtl/fifo.sv
scripts/run_test.sh
tb/tb_fifo.sv
tests/testlist
$ scripts/run_test.sh fifo_basic
PASS  fifo_basic  @ nightly_fail_1017
$ EXTRA=+WAVES scripts/run_test.sh fifo_basic     # also dump waves
PASS  fifo_basic  @ nightly_fail_1017
```

Track everything a person writes: RTL, testbench, sequences, assertions, testlists, file lists (`.f`), scripts, Makefiles, configs and docs.

Never track anything a tool generates: build directories, waveforms, coverage databases, logs, simulator libraries. A single FSDB can be gigabytes, and Git keeps every version of every file forever, so one committed waveform makes every future clone slower for the whole team.

```
$ git status --short
$ git status --short --ignored
!! sim/
$ git check-ignore -v sim/waves.vcd
.gitignore:2:sim/       sim/waves.vcd
```

`git status --short` printed nothing: the repo is clean even though the simulation just created `sim/`. With `--ignored`, `!!` shows what is being ignored, and `git check-ignore -v` names the exact `.gitignore` rule (line 2, `sim/`).

A starting `.gitignore` for a DV repo:

```
# build and work areas
work/
csrc/
simv*
xcelium.d/
INCA_libs/
# waveforms
*.vcd
*.fsdb
*.wlf
*.shm/
# coverage databases
*.ucdb
cov_work/
# logs and run output
*.log
sim/
```

05 Reproduce any result exactly

A DV result is only useful if someone else can get the same result. That needs five things: **the commit, the test, the seed, the tool version and the command line**. The run script records them in every log:

```
$ scripts/run_test.sh fifo_full
FAIL  fifo_full  @ nightly_fail_1017   (log: sim/logs/fifo_full_1.log)
$ head -n 3 sim/logs/fifo_full_1.log
commit: nightly_fail_1017
test: fifo_full  seed: 1
Icarus Verilog version 12.0 (stable) ()
```

`git describe --always --dirty --tags` names the code in a human way:

OUTPUT	MEANING
`nightly_fail_1017`	Exactly the commit tagged `nightly_fail_1017`
`nightly_pass_1015-2-g7c75b56`	2 commits after that tag, at commit `7c75b56`
`nightly_fail_1017-dirty`	That commit **plus uncommitted edits**: nobody else can reproduce this result

```
$ echo "// my local edit" >> tb/tb_fifo.sv
$ scripts/run_test.sh fifo_basic
PASS  fifo_basic  @ nightly_fail_1017-dirty
$ git restore tb/tb_fifo.sv
```

> Never report a failure (or a pass) from a `-dirty` run. Commit or stash first, then rerun.

Worktrees: look at another version without touching your work. `git worktree add` checks out a second copy of the repo in another folder. It shares the same history, and your branch stays exactly as it is:

```
$ git worktree add ../fifo_good nightly_pass_1015
Preparing worktree (detached HEAD 097ad87)
HEAD is now at 097ad87 tb: add fifo_empty test
$ cd ../fifo_good && scripts/run_test.sh fifo_full && cd ../fifo_dv
PASS  fifo_full  @ nightly_pass_1015
$ git worktree list
/home/abishek/fifo_dv    33dfeb9 [main]
/home/abishek/fifo_good  097ad87 (detached HEAD)
$ git worktree remove ../fifo_good
```

The old passing code really passes `fifo_full`. Use worktrees to rerun a nightly failure while you keep working, or to open good and bad waveforms side by side. Use them instead of a second `git clone`, which downloads the whole history again.

06 Find what broke the regression

The 15 Oct nightly passed; the 17 Oct nightly failed `fifo_full`. Three steps find the cause.

Step 1: what changed between the two runs? `A..B` means "commits in B that are not in A":

```
$ git log --oneline nightly_pass_1015..nightly_fail_1017
33dfeb9 rtl: add port comments
64ef749 scripts: record simulator version in each log
7c75b56 rtl: simplify full flag logic
ade5d98 docs: add README with run instructions
$ git diff --stat nightly_pass_1015 nightly_fail_1017
 README.md           |  6 ++++++
 rtl/fifo.sv         | 10 +++++-----
 scripts/run_test.sh |  1 +
 3 files changed, 12 insertions(+), 5 deletions(-)
```

Four suspects in three files. Here you could read them all, but in a real project a day can bring 50 commits.

Step 2: let Git find the guilty commit with `git bisect`. You give one bad and one good point. Git checks out the commit in the middle, you test it, and every answer halves the suspects: 1000 commits take only about 10 simulations. With `git bisect run`, Git runs your script at each step and reads its exit code: **0 = good, 1 to 127 = bad, 125 = skip** (the script returns 125 when the code doesn't build, so Git just tries a neighbouring commit).

```
$ git bisect start nightly_fail_1017 nightly_pass_1015
Bisecting: 1 revision left to test after this (roughly 1 step)
[7c75b5683bae828612c7410ce507d908d7ae3d74] rtl: simplify full flag logic
$ git bisect run scripts/run_test.sh fifo_full
running 'scripts/run_test.sh' 'fifo_full'
FAIL  fifo_full  @ nightly_pass_1015-2-g7c75b56   (log: sim/logs/fifo_full_1.log)
Bisecting: 0 revisions left to test after this (roughly 0 steps)
[ade5d98610b813186644a1132eba2e843acf432d] docs: add README with run instructions
running 'scripts/run_test.sh' 'fifo_full'
PASS  fifo_full  @ nightly_pass_1015-1-gade5d98
7c75b5683bae828612c7410ce507d908d7ae3d74 is the first bad commit
commit 7c75b5683bae828612c7410ce507d908d7ae3d74
Author: Priya Raman <priya@example.com>
Date:   Fri Oct 16 14:20:00 2026 +0530

    rtl: simplify full flag logic

 rtl/fifo.sv | 2 +-
 1 file changed, 1 insertion(+), 1 deletion(-)
bisect found first bad commit
$ git bisect reset
Previous HEAD position was ade5d98 docs: add README with run instructions
Switched to branch 'main'
Your branch is up to date with 'origin/main'.
```

Two simulations, and Git names the commit, its author and the file it changed. `git bisect reset` returns you to where you started.

Step 3: read the change.

```
$ git show 7c75b56
```

```
commit 7c75b5683bae828612c7410ce507d908d7ae3d74
Author: Priya Raman <priya@example.com>
Date:   Fri Oct 16 14:20:00 2026 +0530

    rtl: simplify full flag logic

diff --git a/rtl/fifo.sv b/rtl/fifo.sv
index 862185e..857483d 100644
--- a/rtl/fifo.sv
+++ b/rtl/fifo.sv
@@ -13,7 +13,7 @@ module fifo #(parameter int DEPTH = 4, parameter int WIDTH = 8) (
   logic [$clog2(DEPTH)-1:0] wptr, rptr;
   logic [$clog2(DEPTH):0]   count;

-  assign full  = (count == DEPTH);
+  assign full  = (count == DEPTH-1);
   assign empty = (count == 0);
   assign rdata = mem[rptr];
```

Now the bug is obvious: `full` is raised at 3 entries instead of 4, so the 4th write is silently dropped. That matches the log, which showed `ERROR: exp=a3 act=xx`.

> Bisect with the shortest test that fails. When a step needs a human check (looking at a waveform, for example), run `git bisect good` or `git bisect bad` by hand instead of `git bisect run`.

07 Who changed this line, and why

```
$ git blame -L 16,17 rtl/fifo.sv
```

```
7c75b568 (Priya Raman 2026-10-16 14:20:00 +0530 16)   assign full  = (count == DEPTH-1);
^43d90c9 (Priya Raman 2026-10-12 10:00:00 +0530 17)   assign empty = (count == 0);
```

Each line shows the commit, author and date that last changed it. A `^` marks a line unchanged since the very first commit.

```
$ git log -S "DEPTH-1" --oneline
```

```
7c75b56 rtl: simplify full flag logic
```

`git log -S "text"` (the "pickaxe") lists the commits that added or removed that text. It's perfect for questions like "when did this constraint become `len < 16`?" or "who removed this assertion?".

```
$ git log --format="%h %ad %<(13)%an %s" --date=short -- rtl/fifo.sv
```

```
33dfeb9 2026-10-17 Priya Raman   rtl: add port comments
7c75b56 2026-10-16 Priya Raman   rtl: simplify full flag logic
43d90c9 2026-10-12 Priya Raman   rtl: add synchronous fifo
```

ALSO USEFUL	SHOWS
`git log -p -- tb/tb_fifo.sv`	Every change to one file, with the diff
`git log --grep="BUG-1234"`	Commits whose message mentions a bug ID
`git show <id>`	One commit: message and full diff

> Use blame to find who to **talk to**, not who to blame. Ask why the change was made; there may be a spec reason you don't know.

08 Fix it on a branch and open a Merge Request

Never fix directly on `main`. Create a short branch named after the job (`fix/...`, `tb/...`, `rtl/...`), one topic per branch. `git revert` makes a **new** commit that exactly undoes a bad one. That's safe on shared history, because nothing is deleted.

```
$ git switch -c fix/fifo_full_flag
Switched to a new branch 'fix/fifo_full_flag'
$ git revert --no-edit 7c75b56
Auto-merging rtl/fifo.sv
[fix/fifo_full_flag 147186d] Revert "rtl: simplify full flag logic"
 Date: Sun Oct 18 09:30:00 2026 +0530
 1 file changed, 1 insertion(+), 1 deletion(-)
$ for t in $(cat tests/testlist); do scripts/run_test.sh "$t"; done
PASS  fifo_basic  @ nightly_fail_1017-1-g147186d
PASS  fifo_full  @ nightly_fail_1017-1-g147186d
PASS  fifo_empty  @ nightly_fail_1017-1-g147186d
```

`nightly_fail_1017-1-g147186d` shows the regression ran on the tag plus your one fix commit. Before pushing, check exactly what your Merge Request will contain:

```
$ git log --oneline main..HEAD
147186d Revert "rtl: simplify full flag logic"
$ git push -u origin fix/fifo_full_flag
remote:
remote: To create a merge request for fix/fifo_full_flag, visit:
remote:   https://gitlab.example.com/dv/fifo_dv/-/merge_requests/new?merge_request%5Bsource_branch%5D=fix%2Ffifo_full_flag
remote:
To https://gitlab.example.com/dv/fifo_dv.git
 * [new branch]      fix/fifo_full_flag -> fix/fifo_full_flag
branch 'fix/fifo_full_flag' set up to track 'origin/fix/fifo_full_flag'.
```

GitLab replies with a link to open the Merge Request (MR). A good DV MR description says:

```
What:   Revert "rtl: simplify full flag logic" (7c75b56)
Why:    full was raised at DEPTH-1, so the last write was dropped.
        Found by nightly 17 Oct, fifo_full; git bisect pointed at 7c75b56.
Tested: full testlist passes (3/3) at nightly_fail_1017 + this fix.
Review: @priya (RTL owner)
```

09 Commit only what you mean: git add -p

You add a new test, and while debugging you also drop a `$display` into the testbench. The test should be committed; the debug print should not. `git add -p` shows each change ("hunk") and asks, one at a time:

```
$ git switch main
```

```
Switched to branch 'main'
Your branch is up to date with 'origin/main'.
```

```
$ git pull
```

```
From https://gitlab.example.com/dv/fifo_dv
   33dfeb9..507f8e9  main       -> origin/main
Updating 33dfeb9..507f8e9
Fast-forward
 rtl/fifo.sv | 2 +-
 1 file changed, 1 insertion(+), 1 deletion(-)
```

```
$ git switch -c tb/fifo_overflow_test
```

```
Switched to a new branch 'tb/fifo_overflow_test'
```

```
$ vim tb/tb_fifo.sv tests/testlist     # add the test (and a quick debug print)
```

```
$ git diff --stat
```

```
 tb/tb_fifo.sv  | 7 +++++++
 tests/testlist | 1 +
 2 files changed, 8 insertions(+)
```

```
$ git add -p tb/tb_fifo.sv
```

```
diff --git a/tb/tb_fifo.sv b/tb/tb_fifo.sv
index 4a27db7..c102d81 100644
--- a/tb/tb_fifo.sv
+++ b/tb/tb_fifo.sv
@@ -13,6 +13,7 @@ module tb_fifo;

   task automatic write(input logic [7:0] d);
     @(negedge clk); wr_en = 1; wdata = d;
+    $display("DBG write %h full=%b", d, full);
     @(negedge clk); wr_en = 0;
   endtask

(1/2) Stage this hunk [y,n,q,a,d,j,J,g,/,e,?]? n
@@ -42,6 +43,12 @@ module tb_fifo;
         @(negedge clk);
         if (!empty) begin errors++; $display("ERROR: empty not set after last read"); end
       end
+      "fifo_overflow": begin
+        for (int i = 0; i <= DEPTH; i++) write(8'hB0 + i);       // one write too many
+        if (!full) begin errors++; $display("ERROR: full not set"); end
+        for (int i = 0; i < DEPTH; i++) read_check(8'hB0 + i);  // the extra write must be dropped
+        if (!empty) begin errors++; $display("ERROR: extra write was not dropped"); end
+      end
       default: begin errors++; $display("ERROR: unknown test %0s", test); end
     endcase
     if (errors == 0) $display("TEST PASSED: %0s", test);
(2/2) Stage this hunk [y,n,q,a,d,K,g,/,e,?]? y
```

Answer `y` to stage a hunk, `n` to skip it, `s` to split it into smaller hunks, `q` to quit and `?` for help. Here the debug print was skipped and the new test was staged.

```
$ git add tests/testlist
$ git status --short
MM tb/tb_fifo.sv
M  tests/testlist
$ git commit -m "tb: add fifo_overflow test"
[tb/fifo_overflow_test 1fd2501] tb: add fifo_overflow test
 2 files changed, 7 insertions(+)
$ git restore tb/tb_fifo.sv          # drop the debug print
$ scripts/run_test.sh fifo_overflow
PASS  fifo_overflow  @ nightly_fail_1017-3-g1fd2501
```

`MM` means the file has **both** staged changes (the test, committed) and unstaged changes (the debug print, then thrown away with `git restore`).

Commit messages: one line, starting with the area, saying what changed. Add a blank line and details if the "why" isn't obvious.

WEAK	STRONG
`fix`	`rtl: raise full at DEPTH, not DEPTH-1`
`updated tb`	`tb: add fifo_overflow test`
`changes`	`sb: compare data at 4KB boundary (BUG-1234)`
`wip`	`testlist: add fifo_reset to nightly`

10 Interrupted? git stash

You are halfway through a change when someone asks "does `fifo_full` pass on main now?". `git stash` puts your unfinished edits on a shelf, so you can switch branches with a clean folder:

```
$ vim tb/tb_fifo.sv     # half-way through a coverage counter...
$ git stash push -m "wip: overflow coverage counter"
Saved working directory and index state On tb/fifo_overflow_test: wip: overflow coverage counter
$ git switch main
Switched to branch 'main'
Your branch is up to date with 'origin/main'.
$ scripts/run_test.sh fifo_full
PASS  fifo_full  @ nightly_fail_1017-2-g507f8e9
$ git switch -
Switched to branch 'tb/fifo_overflow_test'
$ git stash list
stash@{0}: On tb/fifo_overflow_test: wip: overflow coverage counter
$ git stash pop
On branch tb/fifo_overflow_test
Changes not staged for commit:
  (use "git add <file>..." to update what will be committed)
  (use "git restore <file>..." to discard changes in working directory)
        modified:   tb/tb_fifo.sv

no changes added to commit (use "git add" and/or "git commit -a")
Dropped refs/stash@{0} (014f97e98312e23c085db1d84ec6d3f46d1343e7)
```

COMMAND	WHAT IT DOES
`git stash push -m "msg"`	Shelve your changes with a label (add `-u` to include new files)
`git stash list`	Show the shelf
`git stash pop`	Bring the latest back and remove it from the shelf
`git stash apply`	Bring it back but keep a copy on the shelf
`git switch -`	Go back to the previous branch (like `cd -`)

> Stash is for minutes. If the interruption is a long simulation or a day of debug, use a worktree (section 5) instead.

11 Stay in sync and resolve testlist conflicts

While you worked, Karthik added a `fifo_reset` test on `main`. `git fetch` downloads it without touching your files, and `HEAD..origin/main` shows what you are missing:

```
$ git fetch origin
From https://gitlab.example.com/dv/fifo_dv
   507f8e9..6d2cd32  main       -> origin/main
$ git log --oneline HEAD..origin/main
6d2cd32 tb: add fifo_reset test
$ git merge origin/main
Auto-merging tb/tb_fifo.sv
Auto-merging tests/testlist
CONFLICT (content): Merge conflict in tests/testlist
Recorded preimage for 'tests/testlist'
Automatic merge failed; fix conflicts and then commit the result.
```

The testbench merged automatically because you edited different places. The testlist did not: you both added a line at the very end.

```
$ cat tests/testlist
fifo_basic
fifo_full
fifo_empty
<<<<<<< HEAD
fifo_overflow
=======
fifo_reset
>>>>>>> origin/main
```

Keep both tests, delete the three marker lines, then finish the merge with a clear message **and rerun the regression**: a merge that compiles is not automatically correct.

```
$ vim tests/testlist     # keep both new tests, delete the marker lines
$ git add tests/testlist
$ git commit -m "Merge origin/main into tb/fifo_overflow_test"
Recorded resolution for 'tests/testlist'.
[tb/fifo_overflow_test 0efce8d] Merge origin/main into tb/fifo_overflow_test
$ for t in $(cat tests/testlist); do scripts/run_test.sh "$t"; done
PASS  fifo_basic  @ nightly_fail_1017-5-g0efce8d
PASS  fifo_full  @ nightly_fail_1017-5-g0efce8d
PASS  fifo_empty  @ nightly_fail_1017-5-g0efce8d
PASS  fifo_overflow  @ nightly_fail_1017-5-g0efce8d
PASS  fifo_reset  @ nightly_fail_1017-5-g0efce8d
$ git push -u origin tb/fifo_overflow_test
remote:
remote: To create a merge request for tb/fifo_overflow_test, visit:
remote:   https://gitlab.example.com/dv/fifo_dv/-/merge_requests/new?
merge_request%5Bsource_branch%5D=tb%2Ffifo_overflow_test
remote:
To https://gitlab.example.com/dv/fifo_dv.git
 * [new branch]      tb/fifo_overflow_test -> tb/fifo_overflow_test
branch 'tb/fifo_overflow_test' set up to track 'origin/tb/fifo_overflow_test'.
```

`Recorded resolution` is rerere at work. If the same conflict appears again, for example after you rebase or redo the merge, Git applies your fix by itself and prints `Resolved 'tests/testlist' using previous resolution.` Shared files like testlists and file lists conflict all the time, so this saves real time.

12 Release branches, cherry-pick and tags

A **release branch** is frozen for a milestone (tapeout, a customer drop). Only approved fixes go in. Here `release/a0` was cut at `nightly_fail_1017`, so it has the bug, and it needs the fix without everything else that landed on `main`. `git cherry-pick` copies exactly one commit:

```
$ git switch release/a0
Switched to a new branch 'release/a0'
branch 'release/a0' set up to track 'origin/release/a0'.
$ scripts/run_test.sh fifo_full
FAIL  fifo_full  @ nightly_fail_1017   (log: sim/logs/fifo_full_1.log)
$ git cherry-pick -x 147186d
[release/a0 73591c9] Revert "rtl: simplify full flag logic"
 Date: Sun Oct 18 09:30:00 2026 +0530
 1 file changed, 1 insertion(+), 1 deletion(-)
$ git log -1 --format=%B
Revert "rtl: simplify full flag logic"

This reverts commit 7c75b5683bae828612c7410ce507d908d7ae3d74.

(cherry picked from commit 147186d5cf5ec19cde13358a6b96b547f740865e)
$ scripts/run_test.sh fifo_full
PASS  fifo_full  @ nightly_fail_1017-1-g73591c9
```

`-x` adds `(cherry picked from commit ...)` to the message. Months later, anyone can trace the release fix back to the original. Then mark exactly what was signed off with an annotated tag:

```
$ git tag -a a0_rc1 -m "A0 release candidate 1: regression clean"
$ git push origin release/a0 a0_rc1
To https://gitlab.example.com/dv/fifo_dv.git
   33dfeb9..73591c9  release/a0 -> release/a0
 * [new tag]         a0_rc1 -> a0_rc1
$ git describe
a0_rc1
```

Tag names your team will recognise: `nightly_pass_1015`, `a0_rc1`, `signoff_a0`. `git describe` printing exactly `a0_rc1` proves the code is the tagged code.

13 Undo safely, and the reflog rescue

SITUATION	COMMAND
Throw away edits to a file	`git restore file`
Unstage a file, keep the edit	`git restore --staged file`
Fix the last commit message (not pushed yet)	`git commit --amend -m "new message"`
Undo a commit that is already pushed	`git revert <id>`
Undo the last local commit, keep its changes	`git reset --soft HEAD~1`
Throw away commits and edits	`git reset --hard <where>`: dangerous

Everyone runs the wrong `reset --hard` one day:

```
$ git switch tb/fifo_overflow_test
Switched to branch 'tb/fifo_overflow_test'
Your branch is up to date with 'origin/tb/fifo_overflow_test'.
$ git log --oneline -3
0efce8d Merge origin/main into tb/fifo_overflow_test
6d2cd32 tb: add fifo_reset test
1fd2501 tb: add fifo_overflow test
$ git reset --hard HEAD~2          # oops: meant HEAD~1, or not at all
HEAD is now at 507f8e9 Merge branch 'fix/fifo_full_flag' into 'main'
$ git log --oneline -3
507f8e9 Merge branch 'fix/fifo_full_flag' into 'main'
147186d Revert "rtl: simplify full flag logic"
33dfeb9 rtl: add port comments
```

Two commits seem gone. They aren't: the **reflog** records every place `HEAD` has been.

```
$ git reflog -4
507f8e9 HEAD@{0}: reset: moving to HEAD~2
0efce8d HEAD@{1}: checkout: moving from release/a0 to tb/fifo_overflow_test
73591c9 HEAD@{2}: cherry-pick: Revert "rtl: simplify full flag logic"
33dfeb9 HEAD@{3}: checkout: moving from tb/fifo_overflow_test to release/a0
$ git reset --hard HEAD@{1}
HEAD is now at 0efce8d Merge origin/main into tb/fifo_overflow_test
$ git log --oneline -3
0efce8d Merge origin/main into tb/fifo_overflow_test
6d2cd32 tb: add fifo_reset test
1fd2501 tb: add fifo_overflow test
```

`HEAD@{1}` means "where HEAD was one move ago". Committed work stays recoverable for weeks (by default, 30 days for commits no branch points to, 90 days otherwise).

> Uncommitted changes destroyed by `reset --hard` are gone for good. Commit or stash before anything risky.

14 Search the code fast: git grep

```
$ grep -rn "fifo_overflow" --exclude-dir=.git .
./sim/logs/fifo_overflow_1.log:2:test: fifo_overflow  seed: 1
./sim/logs/fifo_overflow_1.log:4:TEST PASSED: fifo_overflow
./tb/tb_fifo.sv:49:      "fifo_overflow": begin
./tests/testlist:4:fifo_overflow
$ git grep -n "fifo_overflow"
tb/tb_fifo.sv:49:      "fifo_overflow": begin
tests/testlist:4:fifo_overflow
```

`grep -r` also matched the simulation log. `git grep` searches only files Git tracks, so build output, logs and waveforms never clutter the results. It's also faster on big repos.

EXAMPLE	FINDS
`git grep -n "uvm_error" -- '*.sv'`	Every `uvm_error` in SystemVerilog files
`git grep -nw "full" -- rtl/`	The whole word `full`, in RTL only
`git grep -n "TODO" nightly_pass_1015`	Search the code as it was at a tag

15 Team-scale topics

Submodules. Big DV repos often include other repos: the RTL, a VIP, a UVM library. A submodule pins the exact commit used, which keeps results reproducible.

```
git clone --recurse-submodules https://gitlab.example.com/dv/soc_dv.git
git submodule update --init --recursive     # after a pull that moved a submodule
git submodule status                        # which commit each submodule is pinned to
```

CI pipelines. GitLab can build and run a smoke regression on every Merge Request, so a broken change is caught before it reaches `main`. This pipeline works with the example project: it passes on the fixed code and fails the job (exit 1) on `nightly_fail_1017`.

```
# .gitlab-ci.yml: GitLab runs this on every merge request and on main
stages: [build, regress]

build:
  stage: build
  script:
    - iverilog -g2012 -o simv rtl/fifo.sv tb/tb_fifo.sv

smoke_regression:
  stage: regress
  script:
    - fail=0
    - for t in $(cat tests/testlist); do scripts/run_test.sh "$t" || fail=1; done
    - exit $fail
  artifacts:
    when: always
    paths: [sim/logs/]
  rules:
    - if: $CI_PIPELINE_SOURCE == "merge_request_event"
    - if: $CI_COMMIT_BRANCH == $CI_DEFAULT_BRANCH
```

At large companies the jobs usually go to a compute farm with licensed simulators, but the idea is the same: no merge without a green smoke regression.

Large files. Keep big binaries out of Git. Logs, waveforms and coverage databases belong in the regression system's storage or CI artifacts, as above. Teams that must version large binaries use Git LFS.

16 If your team uses Perforce

GIT	PERFORCE (P4)	NOTE
`git clone`	create a client (workspace), then `p4 sync`	
`git pull`	`p4 sync`	Get the latest files
`git add` / edit	`p4 add` / `p4 edit`	P4 wants you to open a file before editing it
`git commit` + `git push`	`p4 submit`	One step; the result is a numbered **changelist**
`git stash`	`p4 shelve`	Shelved changes live on the server and others can unshelve them (often used for review)
`git blame`	`p4 annotate`	
`git log file`	`p4 filelog`, `p4 changes`	
`git revert <id>`	`p4 undo`	
`git tag`	`p4 label`, or simply a changelist number	
Branches, Merge Requests	Streams, `p4 integrate`; Helix Swarm reviews	

Bisect, reproducibility and clean commits work the same way: "changelist 12345 passed, 12390 fails" is the same search as section 6.

17 Interview questions you can now answer

"Last night's regression passed, today a test fails. What do you do?" Reproduce it first, with the same commit, test, seed and tool version. List what changed between the two runs (`git log pass_tag..fail_tag`). Bisect with the failing test (`git bisect run`), read the guilty diff, and talk to its author. Then fix or revert on a branch, rerun the regression and open an MR.

"How do you make a failure reproducible for a designer?" Give the commit ID (never from a `-dirty` run), the test name, the seed, the tool version and the exact command line. Tag the commit if it matters later.

"How do you undo a commit that is already pushed?" `git revert <id>`. It adds a new commit and doesn't rewrite history that others already have.

"Merge or rebase?" A merge keeps the true history and is always safe on shared branches. A rebase gives a straight line but changes commit IDs, so only rebase commits that nobody else has.

"Two engineers added a test to the same testlist. What happens?" A merge conflict on the lines both changed. Keep both tests, remove the markers, rerun the regression, then commit. `rerere` remembers the fix.

"Why not commit logs and waveforms?" They are generated, huge and different on every run. Git keeps every version forever, so the repo would grow without limit. They belong in regression or CI storage.

> A strong portfolio shows these habits: in your own repos, use branches and Merge or Pull Requests even when working alone, tag milestones, keep a CI regression green and write clear commit messages.

18 Daily cheat sheet

The whole story in one graph: the fix merged to `main`, your test branch merged with Karthik's work, and the fix cherry-picked onto the release branch.

```
$ git lg -12
```

```
* 73591c9 (tag: a0_rc1, origin/release/a0, release/a0) Revert "rtl: simplify full flag logic"
| *   0efce8d (HEAD -> tb/fifo_overflow_test, origin/tb/fifo_overflow_test) Merge origin/main into tb/fifo_overflow_test
| |\
| | * 6d2cd32 (origin/main, origin/HEAD) tb: add fifo_reset test
| * | 1fd2501 tb: add fifo_overflow test
| |/
| * 507f8e9 (main) Merge branch 'fix/fifo_full_flag' into 'main'
|/|
| * 147186d (origin/fix/fifo_full_flag, fix/fifo_full_flag) Revert "rtl: simplify full flag logic"
|/
* 33dfeb9 (tag: nightly_fail_1017) rtl: add port comments
* 64ef749 scripts: record simulator version in each log
* 7c75b56 rtl: simplify full flag logic
* ade5d98 docs: add README with run instructions
* 097ad87 (tag: nightly_pass_1015) tb: add fifo_empty test
* 95e290c tb: add fifo_full test
```

SITUATION	COMMAND
Where am I, what changed?	`git st`, `git lg -15`
Start a task	`git switch main`, `git pull`, `git switch -c tb/my_test`
Stage only part of a file	`git add -p file`
What will my MR contain?	`git log --oneline main..HEAD`, `git diff --stat main...HEAD`
First push of a branch	`git push -u origin tb/my_test`
What's new on main?	`git fetch`, `git log --oneline HEAD..origin/main`
Bring main into my branch	`git merge origin/main`
Name this exact code	`git describe --always --dirty --tags`
What changed between two regressions?	`git log --oneline passA..failB`, `git diff --stat passA failB`
Find the commit that broke a test	`git bisect start BAD GOOD`, `git bisect run scripts/run_test.sh test`
Who changed this line?	`git blame -L 10,20 file`
When did this text appear or disappear?	`git log -S "text" --oneline`
Look at old code without losing my work	`git worktree add ../old <tag-or-id>`
Park unfinished work	`git stash push -m "msg"`, later `git stash pop`
Undo a pushed commit	`git revert <id>`
Copy one fix to a release branch	`git cherry-pick -x <id>`
Mark what was signed off	`git tag -a name -m "msg"`, `git push origin name`
I lost commits!	`git reflog`, then `git reset --hard HEAD@{n}`
Search source only	`git grep -n "text" -- '*.sv'`